

46-927: Homework 1

Solutions

Note: at the top we call a lot of the packages we expect to use.

```
In [1]: # needed packages
import numpy as np
import pandas as pd

import scipy.stats

from sklearn import linear_model
import statsmodels.api as sm
import statsmodels.formula.api as smf

from progressbar import ProgressBar

import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
```

```
In [2]: scipy.random.seed(927)
```

Problem 1

a) and b)

```
In [3]: X = scipy.stats.expon(scale = 1).rvs(1000)
# ^ Note: must use scale = 1, since first parameter is loc (annoyingly so)

# values for true distribution
x_mu = 1
x_median = np.log(2)

d = {"names":
      ["mean (I):", "median (II):"],
      "mean squared error":
      [np.mean((X-x_mu)**2), np.mean((X-x_median)**2)],
      "mean absolute error":
      [np.mean(np.abs(X-x_mu)), np.mean(np.abs(X-x_median))]}
df = pd.DataFrame(data=d,
                  columns=["names", "mean squared error",
                          "mean absolute error"])
```

```
In [4]: df
```

```
Out[4]:
```

	names	mean squared error	mean absolute error
0	mean (I):	1.008315	0.729795
1	median (II):	1.090603	0.673042

It appears that we have a lower mean squared error when estimating the mean, and a higher mean squared error when estimating the median with the same amount of observations. The opposite is true when using median squared error - that is, we obtain a higher median squared error when estimating the mean compared to the median.

Problem 2

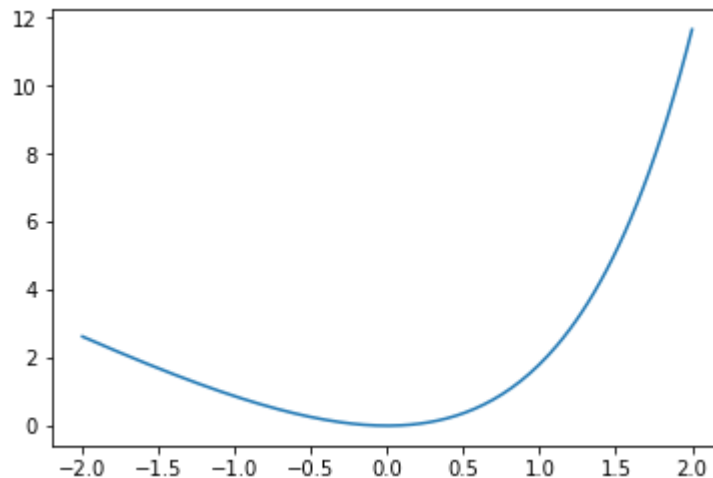
a)

New Loss Function:

$$\mathcal{L}(Y, f(X)) = b(e^{a(Y-f(X))} - a(Y-f(X)) - 1)$$

```
In [5]: def new_loss_function(z, a, b):  
        return( b * (np.exp(a* z) - a*z - 1))
```

```
In [6]: z_range = np.linspace(-2,2,200)  
        plt.plot(z_range, new_loss_function(z_range, a = 1.1, b = 2));
```



This loss function is asymmetric around 0 error. It punishes errors $(Y - f(X))$ in the $\text{sign}(a)$ direction exponentially, and the opposite direction it is just punished linearly. This could be an optimal loss function if we would like to avoid errors on the right side of 0.

b)

Finding the optimal $f(X)$ to minimize expected loss (we'll condition on X).

Solution

$$\begin{aligned}\mathcal{L}(Y, \mu) &= b[e^{a(Y-\mu)} - a(Y - \mu) - 1] \\ \mathbb{E}[\mathcal{L}|X] &= \mathbb{E}[be^{aY}e^{-a\mu} - abY + ab\mu - b|X] \\ &= be^{-a\mu}\mathbb{E}[e^{aY}|X] - \underline{ab\mathbb{E}[Y|X]} + ab\mu - \underline{b}\end{aligned}$$

The above underlined quantities don't involve μ , and as such when we optimize w.r.t. μ will be dropped.

$$\frac{d}{d\mu}\mathbb{E}[\mathcal{L}|X] = -abe^{-a\mu}\mathbb{E}[e^{aY}|X] + ab$$

set $\frac{d}{d\mu}\mathbb{E}[\mathcal{L}|X]$ equal to 0:

$$\begin{aligned}-abe^{-a\mu}\mathbb{E}[e^{aY}|X] + ab &= 0 \\ abe^{-a\mu}\mathbb{E}[e^{aY}|X] &= ab \\ e^{-a\mu}\mathbb{E}[e^{aY}|X] &= 1 \\ \frac{1}{a}\ln(\mathbb{E}[e^{aY}|X]) &= \mu \quad \text{via algebra}\end{aligned}$$

So with the above loss function $\hat{\mu} = \frac{1}{a}\ln(\mathbb{E}[e^{aY}|X])$.

c)

Supposing $Y|X = x \sim N(\beta x, \sigma^2)$ with β and σ^2 known positive scalars.

Solution

Giving the above assumptions, $\mathbb{E}[e^{aY}|X] = e^{a\beta X + 1/2\sigma^2 a^2}$. Plugging this into the solution in **part b** we obtain:

$$\begin{aligned}\mu &= \frac{1}{a}\ln(\mathbb{E}[e^{aY}|X]) \\ &= \frac{1}{a}\ln(e^{a\beta X + 1/2\sigma^2 a^2}) \\ &= \beta X + \frac{a\sigma^2}{2}\end{aligned}$$

d)

Looking at $\mathbb{E}[Y|X]$ vs solution in **part c** for the given loss function.

```

In [7]: ### code from asymm_loss.py

from scipy.stats import norm
import numpy as np

#Set some parameters
beta = 0.5
b = 2
sigma = 2
a = 1

#Define the loss function, where z = y - yhat
def loss(z):
    return b*(np.exp(a*z)-a*z-1)

#Estimation functions
#Estimation using the conditional expectation of Y|X
def f_condexp(x):
    return beta*x

#TODO: Put your function in here. You can reference a,b,sigma, and i
t will just pull them from
# the outside namespace
def f_yours(x):
    return (1/a)*(beta*x*a + .5 * (sigma**2)*(a**2))

#Simulation to see how you do
reps = 1000

#Just generate the X variables normally. We don't really care
x = norm.rvs(size=reps, loc=0, scale=1)

#Generate the Y variables from our normal model
y = norm.rvs(size=reps, loc=x*beta, scale=sigma)

#Calculate the fitted values for each method
yhat_condexp = np.apply_along_axis(f_condexp, 0, x)
yhat_yours = np.apply_along_axis(f_yours, 0, x)

#Compute the losses
condexp_losses = np.apply_along_axis(loss, 0, y-yhat_condexp)
your_losses = np.apply_along_axis(loss, 0, y-yhat_yours)

print("Average loss of the conditional expectation:",
      round(np.mean(condexp_losses),2))

print("Average loss of your method:",
      round(np.mean(your_losses),2))

```

```

Average loss of the conditional expectation: 8.69
Average loss of your method: 3.56

```

The average simulated loss for the solution derived in **part c** is much lower than the conditional expectation. Since the $\mathcal{L}$ is exponential in the positive side if $Y - f(X) > 0$, it makes more sense to estimate values $f(X)$ that are larger Y to have the less harsh penalty (just linear).

Problem 3

```
In [8]: def generate_data_problem2(size=20, p=1):
        """
        size: number of observations
        p: number of dimensions of X
        """
        mvnorm = scipy.stats.multivariate_normal(mean=[0]*p,
                                                    cov=np.diag(np.ones(p)))

        X = mvnorm.rvs(size=size).reshape(size,p)
        y = 4 * X[:,0] + scipy.stats.norm().rvs(size=size)

        return X, y
```

i), ii), and iii)

```
In [9]: p_range = np.arange(80) + 1
mse_mat = np.zeros((p_range.shape[0], 100))

bar = ProgressBar()
for p_count, p in bar(list(enumerate(p_range))):
    # ^this is kinda ugly (feel free to remove bar(list()))
    for sim in np.arange(100):
        X_train, y_train = generate_data_problem2(size=100, p=p)

        X_test, y_test = generate_data_problem2(size=100, p=p)
        lm_out = sm.OLS(y_train,X_train,hasconst = True).fit()
        # ^ in this setup (with `hasconst = True`), it makes it so th
at
        # no beta_0 is fit (the working of the problem could be read
either way)
        y_pred_test = lm_out.predict(exog=X_test)

        mse_mat[p_count,sim] = np.mean((y_test - y_pred_test)**2)

100% (80 of 80) |#####| Elapsed Time: 0:03:33 Tim
e: 0:03:33
```

Aside:

When I know code is going to take a while, I really enjoy using `progressbar2`'s progress bar.

Note the "ugly" coding is because objects from `enumerate(.)` and things like `range(5)` are iterators in python, and don't have a `length` -per-se. Feel free to ask about this in OHs.

```
In [10]: mean_pred_errors = np.mean(mse_mat,axis = 1)
```

Requested Graphic

```
In [11]: fig, ax = plt.subplots()
ax.set(xlabel="p (Dimension of X)",
      ylabel="MSE of model on test data",
      title="MSE across dimensions")
ax.plot(p_range, mean_pred_errors, "k-");
```

